

Agency App Backend — Bug Fix Report

Node.js / Express / MySQL (mysql2) backend — fixes applied following the code audit.

*Scope note: authentication/authorization was intentionally **not** implemented in this pass (the current frontend sends no token; enabling it now would break every existing request) — tracked as a separate follow-up. The invoice-number generation logic/format itself was left untouched per instruction, as it was implemented to a specific client requirement.*

#	File	What Was Fixed	Category	Logic Change?
1	userController.js / SalaryController.js	Removed password_hash from every API response (getUsers, getUserById, searchUser, getAllMarketingStaff).	Security	No
2	Multiple controllers	Removed console.log calls that printed plaintext passwords, password hashes, and customer PII to server logs (login and 10+ other endpoints).	Security	No
3	purchaseController.js (getAllPurchasesByValues)	Fixed SQL injection: limit/page are now parsed with parseInt and LIMIT/OFFSET use ? placeholders instead of raw string interpolation.	Security	No
4	userController.js, productPriceController.js, salesGstReportController.js	Stopped returning raw internal error objects/messages to API clients; generic error messages returned instead.	Security	No
5	productController.js (addProduct)	Input validation now happens before beginTransaction(). Previously an early return left an open transaction on a connection returned to the pool.	Database & Transactions	No
6	productPriceController.js (updateProductPrice)	Same class of fix: validate first; every early-return path now rolls back before releasing the connection.	Database & Transactions	No
7	userController.js (updateUserAccess)	Replaced an invalid pool.rollback() call (method doesn't exist on a pool) with a real connection transaction wrapping the whole batch of menu-access updates.	Database & Transactions	Yes – edge case only: updates are now all-or-nothing. Previously a mid-batch failure could leave some rows updated and others not.
8	productController.js (addMiscDamagedProduct)	Wrapped the insert + stock update in one transaction instead of two independent, unguarded queries.	Database & Transactions	Yes – edge case only: now atomic. Previously a failure between the two writes could leave the damage log and stock count out of sync.
9	dailyStockAllocationController.js (addDailyStockAllocation)	Wrapped the loop of per-item inserts in one transaction.	Database & Transactions	Yes – edge case only: now atomic. Previously a mid-loop failure could leave a partial set of allocations inserted.
10	salesController.js (addSales)	Wrapped insert-sale / update-stock / insert-history in one transaction (previously 3 unguarded, independent queries).	Database & Transactions	Yes – edge case only: previously a stock-update failure still left the sale row inserted (orphaned data). Now the whole request rolls back.
11	purchaseController.js (createEnhancedPurchase, createPurchaseNew)	Added FOR UPDATE row lock on the damaged-stock check before decrementing it.	Database & Transactions	Yes – concurrency only: prevents two simultaneous requests from both passing the stock check and driving Damage_Qty negative. Single-request behavior is unchanged.
12	salesController.js (addSalesFromDailyAllocation)	Added FOR UPDATE row lock on the invoice-counter read (it already existed in addDirectSales but was missing here).	Database & Transactions	Yes – concurrency only: prevents duplicate invoice numbers under simultaneous requests. Invoice number format/algorithm unchanged.
13	purchaseController.js (createEnhancedPurchase)	Moved the activity-log (audit trail) write to after connection.commit() instead of mid-transaction.	Database & Transactions	Yes – edge case only: a log entry is no longer written for a purchase that later rolled back. No change when the purchase succeeds.

#	File	What Was Fixed	Category	Logic Change?
14	cron/dailyOpeningClosingCr on.js	The "no active products" early return now commits the (empty) transaction before returning, instead of abandoning it open.	Database & Transactions	No – only affects an edge case with zero active products.
15	config/db.js	Added a check that fails fast with a clear error if DB_HOST/DB_USER/DB_PASS/DB_NAME are missing, plus a pool-level error handler.	Database & Transactions	No
16	salesController.js (addDirectSales, fetchAllCreditReport)	Moved request-body destructuring/validation and pool.getConnection() inside try/catch. Previously a malformed request body threw <i>before</i> any try block, which can crash the whole Node process on modern Node defaults.	Async & Error Handling	No – only changes the failure path (now returns 400/500 JSON instead of crashing).
17	salesGstReportController.js (fetchGstReport)	Same fix as above: body destructuring moved inside try/catch.	Async & Error Handling	No – failure path only.
18	dailyStockAllocationControlle r.js	Removed a dead, duplicate addDailyStockAllocation function definition that was silently shadowed and never executed.	Correctness & Robustness	No – the removed code never ran in the first place.
19	dailyStockAllocationControlle r.js (deleteDailyStockAllocation)	Fixed the response message, which claimed "stock updated successfully" while stock is (correctly) never touched — allocated_quantity is only a virtual reservation, never physically subtracted, so nothing needs adding back.	Correctness & Robustness	No – wording only; the underlying (correct) behavior was left as-is.
20	salesController.js (addDirectSales)	Fixed the invoice-counter update using WHERE id = 1 instead of WHERE isActive = 1 (inconsistent with every other counter update in the same function).	Correctness & Robustness	Possible, data-dependent: only changes anything if the active counter row's id isn't 1. Invoice number format/generation algorithm was not changed.
21	SalaryController.js (updateStaffSalary)	Added a check for a missing AddedDate before calling .split() on it (previously crashed with an unclear 500 error).	Correctness & Robustness	No – only changes the error returned for invalid input.
22	cashFlowController.js (fetchProfitLoss)	Added default values for page/limit (its sibling endpoint already had them); previously missing values silently produced NaN pagination.	Correctness & Robustness	No – only affects requests that omit page/limit.
23	UserController.js (editUser)	Removed a duplicate, unused bcrypt hash computation (the password was being hashed twice per edit, once discarded).	Correctness & Robustness	No – pure dead-code removal.
24	salesController.js (getGSTInvoiceNumber / getNGSTInvoiceNumber)	Simplified from an unnecessary open-transaction-then-commit around a single read-only SELECT to a plain query.	Correctness & Robustness	No – same value computed and returned.

Did any business logic change?

Short answer: No core business logic changed. Nothing about pricing, GST/commission calculations, invoice number format/algorithm, stock accounting rules, or any business decision the app makes for a normal, successful request was altered.

What did change, and why it was necessary

- **Failure-path behavior (transactions)** — Several endpoints previously ran multiple related database writes as independent, unguarded queries with no transaction. If one write failed partway through, the earlier writes stayed committed — e.g. a sale row could exist with no matching stock deduction. These are now wrapped in real transactions, so on failure everything rolls back instead of leaving partial data. This only changes what happens when something goes wrong; a normal successful request produces the exact same data as before.
- **Failure-path behavior (crash prevention)** — A few endpoints read/validated the request body before any try/catch existed, so a malformed request could throw an unhandled error and, on current Node.js defaults, take down the entire server process. Validation now happens inside try/catch and returns a normal 400/500 JSON response instead. Valid requests are unaffected.
- **Concurrency behavior (FOR UPDATE locks)** — Two places read a shared counter/stock row and then updated it later without locking the row in between. Under concurrent requests this could produce duplicate invoice numbers or a negative damaged-stock count. Row locks were added so only one request at a time can act on that row. A single request running alone behaves identically to before.
- **One data-dependent fix (invoice counter)** — In addDirectSales, one of the two invoice-counter UPDATE statements filtered on **WHERE id = 1** while every other counter update in the same function (and in the rest of the codebase) filters on **WHERE isActive = 1**. This was corrected to **isActive = 1** for consistency. This changes behavior only if the active counter row does not have id = 1. The invoice number format, prefix/suffix logic, and counter increment logic itself were not touched.
- **Response shape (security)** — password_hash is no longer included in API responses that list/search users or marketing staff. Any frontend code that happened to read that field (it shouldn't) will now get undefined instead of the hash.
- **Wording only** — deleteDailyStockAllocation's response message no longer claims stock was updated, since it correctly never was (allocated_quantity is a virtual reservation, not a physical stock deduction — confirmed by checking how stock is computed elsewhere before touching this).

Explicitly not touched

- Invoice number generation algorithm/format (prefix, suffix, padding, GST vs. NGST branching).
- Authentication/authorization — still no auth on any route, left as a deliberate follow-up.
- Any pricing, commission, GST, or stock-accounting formula.

Verification performed: **node --check** on every modified file (all pass), plus a full application boot test (**require('./app.js')**) with dummy DB credentials — server starts cleanly; with no DB credentials set, it now fails immediately with a clear error instead of an unclear runtime failure later.